

— SQL PROJECT — RETAIL STORE

A Comprehensive SQL Project for Data Analysis and Reporting

CASE STUDY COMPANY

LICERIA & CO.

Building Data-Driven Retail Insights



Project Submitted By
Ajit Kumar



Learner ID
24CRD04936



Batch
Data Analytics Course (#01701)



Date
11-July-2026



MySQL Version
8.0.46



Training Organization
Adda247 Skills



Our Mentor
Shaghil Jamal

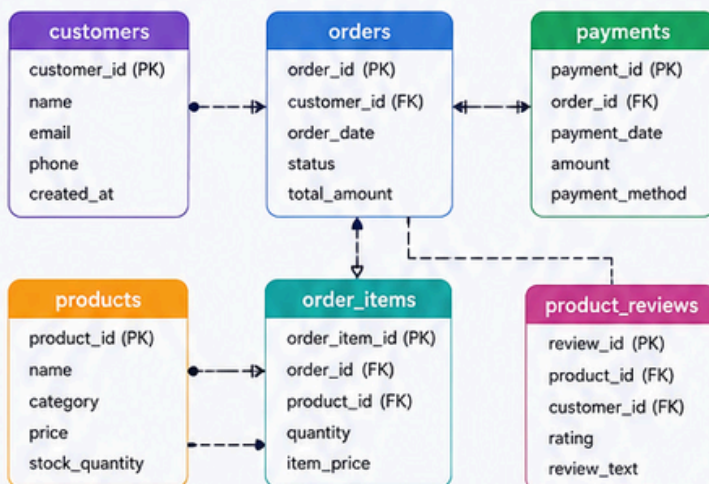
ABOUT THE PROJECT

This project demonstrates the practical implementation of SQL concepts using a retail store database.

It covers database design, data retrieval, filtering, joins, aggregations, subqueries, and set operations through real-world business scenarios.

The project is built on MySQL 8.0 and showcases how SQL can be used to analyze customer, product, order, payment, and review data for business decision-making.

DATABASE SCHEMA



PROJECT HIGHLIGHTS

- ✓ Comprehensive SQL Queries from Basic to Advanced
- ✓ Real-world Retail Business Scenarios
- ✓ Data Analysis and Reporting
- ✓ Joins, Subqueries, Aggregations and Set Operations
- ✓ Clean, Well-Structured and Optimized Queries
- ✓ Insights for Better Business Decisions



Good data isn't just numbers. It tells the story of smarter business.

Level 1: Basics

1. Retrieve customer names and emails for email marketing

SQL Query

```
select  
name,email  
from customers;
```

OUTPUT

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	name	email			
▶	Thomas Owens	user1@example.com			
	Charles Grant	user2@example.com			
	Kaitlin Richards	user3@example.com			
	Christina Williams	user4@example.com			
	David Allen	user5@example.com			
	Mark Duke	user6@example.com			
	Briana Wright	user7@example.com			
	John Bryan	user8@example.com			
	Jason Thompson	user9@example.com			
	Shawn Hill	user10@example.com			
	Walter Jenkins	user11@example.com			
	Mary Knight	user12@example.com			
	Leslie Wilson	user13@example.com			
	Deborah Arias	user14@example.com			
	Austin Flores	user15@example.com			
	Amy Landry	user16@example.com			
	Randy Mooney	user17@example.com			
	Jeffrey Bray	user18@example.com			
	Amanda Bright	user19@example.com			
	Megan Lee	user20@example.com			
	Jessica Zamora	user21@example.com			
	Peter Phillips	user22@example.com			
	Kathryn Mathews	user23@example.com			
	Brandy Wright	user24@example.com			
	Cindy Hart	user25@example.com			
	Victoria Hall	user26@example.com			
	Adrienne Green	user27@example.com			

2. View complete product catalog with all available details

SQL Query

```
select *
from products;
```

OUTPUT

Result Grid		Filter Rows:	Edit:		Export/Import:	
	product_id	name	category	price	stock_quantity	added_on
▶	1	Plant No	Home	639.43	152	2024-01-30 06:30:53
	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	14	Listen Development	Home	296.17	132	2024-10-12 11:49:26
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
	21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
	22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
	23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
	24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
	25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
	26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07

3. List all unique product categories

SQL Query

```
select  
distinct category  
from products;
```

OUTPUT

Result Grid	
	category
▶	Home
	Clothing
	Electronics
	Toys
	Books

4. Show all products priced above ₹1,000

SQL Query

```
select *  
from products  
where price > 1000;
```

OUTPUT

Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap C	
	product_id	name	category	price	stock_quantity	added_on			
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50			
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46			
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03			
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37			
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08			
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10			
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57			
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15			
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06			
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38			
	16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40			
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55			
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53			
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01			
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27			
	21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40			
	22	Move Small	Books	4168.08	22	2023-11-12 18:26:30			
	23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13			
	24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17			
	25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43			
	26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07			
	27	Answer But	Home	1016.68	90	2025-04-06 21:54:11			
	28	East Foot	Toys	2414.93	128	2025-05-11 16:02:29			
	29	Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11			
	30	Future Sort	Home	3043.67	100	2025-05-17 06:55:03			
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06			
	32	Factor Where	Clothing	2295.14	167	2023-07-07 00:59:39			
	33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17			

5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)

SQL Query

```
select *  
from products  
where price  
between 2000 and 5000;
```

OUTPUT

	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
	21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
	22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
	23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
	24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
	25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
	26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
	28	East Foot	Toys	2414.93	128	2025-05-11 16:02:29
	29	Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11
	30	Future Sort	Home	3043.67	100	2025-05-17 06:55:28
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	32	Factor Where	Clothing	2295.14	167	2023-07-07 00:59:39
	33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
	35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
	38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
	39	Data Add	Books	4063.38	62	2025-05-26 23:39:16

6. Fetch data for specific customer IDs (e.g., from loyalty program list)

SQL Query

```
SELECT *
FROM customers
WHERE customer_id IN
(5, 8, 12, 18, 25);
```

OUTPUT

customer_id	name	email	phone	created_at
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
12	Mary Knight	user12@example.com	361-636-3802	2023-08-16 20:05:50
18	Jeffrey Bray	user18@example.com	164.962.0222x92	2024-07-02 19:51:55
25	Cindy Hart	user25@example.com	617.574.8421x41	2025-05-27 18:02:54

7. Identify customers whose names start with the letter 'A

SQL Query

```
SELECT *  
FROM customers  
WHERE name LIKE 'A%';
```

OUTPUT

	customer_id	name	email	phone	created_at
▶	15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
	16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
	19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
	27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29

8. List electronics products priced under ₹3,000

SQL Query

```
SELECT *  
FROM products  
WHERE price < 3000;
```

OUTPUT

	product_id	name	category	price	stock_quantity	added_on
▶	1	Plant No	Home	639.43	152	2024-01-30 06:30:53
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	14	Listen Development	Home	296.17	132	2024-10-12 11:49:26
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
	27	Answer But	Home	1016.68	90	2025-04-06 21:54:11
	28	East Foot	Toys	2414.93	128	2025-05-11 16:02:29
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	32	Factor Where	Clothing	2295.14	167	2023-07-07 00:59:39
	34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
	36	Television Stock	Clothing	421.73	115	2024-04-23 23:54:15
	37	Between Up	Toys	1429.45	61	2025-01-27 01:30:37
	40	Toward Minute	Clothing	857.85	136	2025-04-04 07:04:21
	43	Least Green	Toys	1398.26	81	2024-04-10 15:23:33
	44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
	47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
	50	Stock Article	Home	834.75	131	2024-11-27 15:58:32

9. Display product names and prices in descending order of price

SQL Query

```
SELECT name,price  
FROM products  
order by price desc;
```

OUTPUT

	name	price
▶	Response Indeed	4897.36
	Population Social	4813.68
	Development System	4801.78
	Any Question	4759.28
	Fire Often	4734.89
	Natural Network	4722.66
	Build High	4707.14
	Serious Recognize	4523.10
	Study Total	4413.68
	Real Source	4398.66
	Involve Officer	4244.09
	Group But	4180.23
	Every Amount	4173.60
	Move Small	4168.08
	Drop Remember	4160.45
	Data Add	4063.38
	Action Ask	4017.01
	Weight Enter	3875.18
	Bed Including	3845.31
	Age Treatment	3728.83
	Some Them	3673.86
	Suffer We	3657.43
	South Free	3543.58
	Movement Future	3502.62
	Assume Serve	3437.81
	Rest Something	3077.82
	Future Sort	3043.67
	Available Answer	2529.51

10. Display product names and prices, sorted by price and then by name

SQL Query

```
SELECT name,price  
FROM products  
order by price , name ;
```

OUTPUT

	name	price
►	If Whatever	177.40
	Listen Development	296.17
	Actually Term	396.11
	Television Stock	421.73
	Southern Thing	512.46
	Plant No	639.43
	Place Low	723.97
	Stock Article	834.75
	Toward Minute	857.85
	Common Study	985.19
	Answer But	1016.68
	Special Fact	1094.18
	Despite Win	1340.34
	Least Green	1398.26
	Between Up	1429.45
	Build Her	1852.64
	Series Page	2070.37
	Full West	2112.33
	Life Series	2208.99
	Factor Where	2295.14
	East Foot	2414.93
	Everything Plant	2496.68
	Available Answer	2529.51
	Future Sort	3043.67
	Rest Something	3077.82
	Assume Serve	3437.81
	Movement Future	3502.62
	South Free	3543.58

Level 2: Filtering and Formatting

1. Retrieve orders where customer information is missing (possibly due to data migration or deletion)

SQL Query

```
SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.status,
    o.total_amount
FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```

OUTPUT

Result Grid

Filter Rows:

Export:

Wrap Cell

	order_id	customer_id	order_date	status	total_amount
--	----------	-------------	------------	--------	--------------

2. Display customer names and emails using column aliases for frontend readability

SQL Query

```
SELECT  
    name AS customer_name,  
    email AS email_address  
FROM customers;
```

OUTPUT

	customer_name	email_address
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com
	Shawn Hill	user10@example.com
	Walter Jenkins	user11@example.com
	Mary Knight	user12@example.com
	Leslie Wilson	user13@example.com
	Deborah Arias	user14@example.com
	Austin Flores	user15@example.com
	Amy Landry	user16@example.com
	Randy Mooney	user17@example.com
	Jeffrey Bray	user18@example.com
	Amanda Bright	user19@example.com
	Megan Lee	user20@example.com
	Jessica Zamora	user21@example.com
	Peter Phillips	user22@example.com
	Kathryn Mathews	user23@example.com
	Brandy Wright	user24@example.com
	Cindy Hart	user25@example.com
	Victoria Hall	user26@example.com
	Adrienne Green	user27@example.com
	Joseph Stuart	user28@example.com

3. Calculate total value per item ordered by multiplying quantity and item price

SQL Query

```
SELECT
    order_item_id,
    order_id,
    product_id,
    quantity,
    item_price,
    (quantity * item_price) AS total_value
FROM order_items;
```

OUTPUT

	order_item_id	order_id	product_id	quantity	item_price	total_value
▶	1	1	19	2	4707.14	9414.28
	2	2	6	3	177.40	532.20
	3	3	9	3	985.19	2955.57
	4	3	23	1	2208.99	2208.99
	5	4	35	2	4734.89	9469.78
	6	5	19	1	4707.14	4707.14
	7	5	7	2	4897.36	9794.72
	8	6	7	3	4897.36	14692.08
	9	6	37	1	1429.45	1429.45
	10	6	15	1	723.97	723.97
	11	6	35	3	4734.89	14204.67
	12	7	30	1	3043.67	3043.67
	13	8	10	3	4801.78	14405.34
	14	8	5	3	4722.66	14167.98
	15	8	31	2	2070.37	4140.74
	16	9	7	3	4897.36	14692.08
	17	9	44	1	396.11	396.11
	18	9	30	3	3043.67	9131.01
	19	10	37	1	1429.45	1429.45

4. Combine customer name and phone number in a single column

SQL Query

```
SELECT
    customer_id,
    CONCAT(name, ' - ', phone) AS
customer_contact
FROM customers;
```

OUTPUT

Result Grid			Filter Rows:	Export:
	customer_id	customer_contact		
▶	1	Thomas Owens - 142-479-1945		
	2	Charles Grant - 9153947511		
	3	Kaitlin Richards - 2073473421		
	4	Christina Williams - 586-605-5061x06		
	5	David Allen - (751)456-8289x1		
	6	Mark Duke - (144)957-2811		
	7	Briana Wright - 223-833-9635		
	8	John Bryan - 045.568.0798x27		
	9	Jason Thompson - 1862659420		
	10	Shawn Hill - (268)113-3152x7		
	11	Walter Jenkins - 536-329-0817x71		
	12	Mary Knight - 361-636-3802		
	13	Leslie Wilson - +1-256-261-1984		
	14	Deborah Arias - 811-821-2144x97		
	15	Austin Flores - 329.901.1576x66		
	16	Amy Landry - +1-278-019-3748		
	17	Randy Mooney - (158)927-7313x3		
	18	Jeffrey Bray - 164.962.0222x92		
	19	Alexander Smith - 888.881.8788.88		

5. Extract only the date part from order timestamps for date-wise reporting

SQL Query

```
SELECT
    order_id,
    customer_id,
    DATE(order_date) AS order_date
FROM orders
order by order_date;
```

OUTPUT

	order_id	customer_id	order_date
▶	18	13	2024-06-15
	199	30	2024-06-18
	66	27	2024-06-19
	84	28	2024-06-19
	24	6	2024-06-19
	160	18	2024-06-20
	208	7	2024-06-20
	180	6	2024-06-21
	175	30	2024-06-21
	140	16	2024-06-23
	56	2	2024-06-23
	85	21	2024-06-24
	341	8	2024-06-25
	142	17	2024-06-30
	359	10	2024-06-30
	267	23	2024-07-03
	41	20	2024-07-04
	38	30	2024-07-05
	...	...	...

6. List products that do not have any stock left

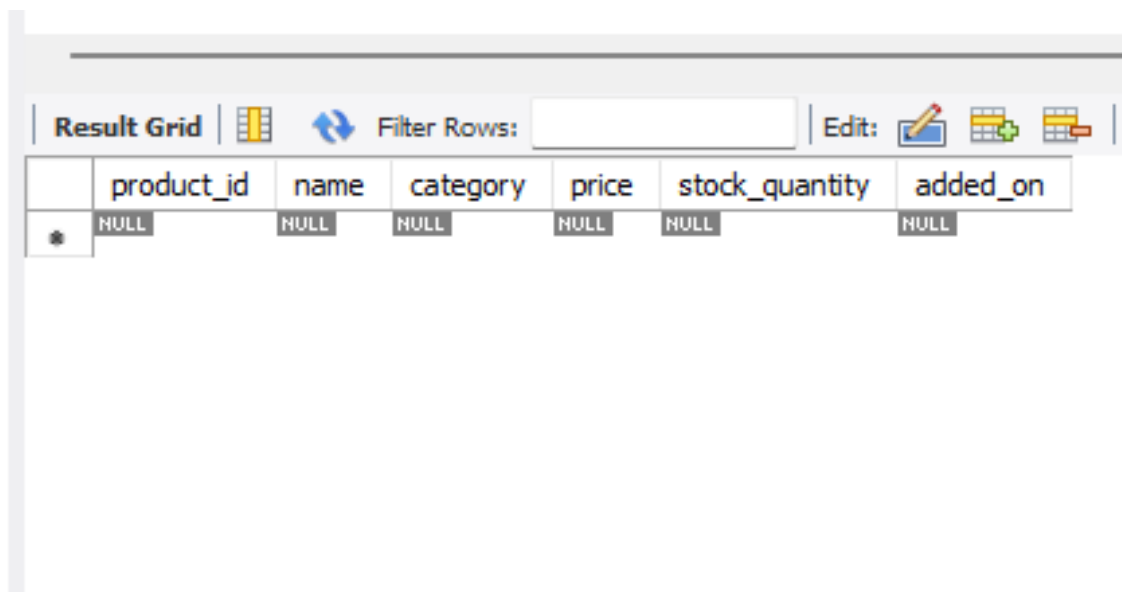
SQL Query

SELECT

FROM products

WHERE stock_quantity = 0;

OUTPUT



The screenshot shows a database application interface. At the top, there is a toolbar with icons for 'Result Grid', 'Filter Rows', and 'Edit'. Below the toolbar is a table with the following columns: product_id, name, category, price, stock_quantity, and added_on. The table contains one row with all NULL values. The first cell of the row contains an asterisk (*).

	product_id	name	category	price	stock_quantity	added_on
*	NULL	NULL	NULL	NULL	NULL	NULL

Level 3: Aggregations

1. Count the total number of orders placed

SQL Query

```
SELECT  
    count(*) as total_order  
FROM orders;
```

OUTPUT

Result Grid	
	total_order
▶	400

2. Calculate the total revenue collected from all orders

SQL Query

```
SELECT  
    sum(total_amount) as Total_Revenue  
FROM orders;
```

OUTPUT

	Total_Revenue
▶	6960973.66

3. Calculate the average order value

SQL Query

```
SELECT  
  round(avg(total_amount) ,2)as 'Average  
Order Value'  
FROM orders;
```

OUTPUT

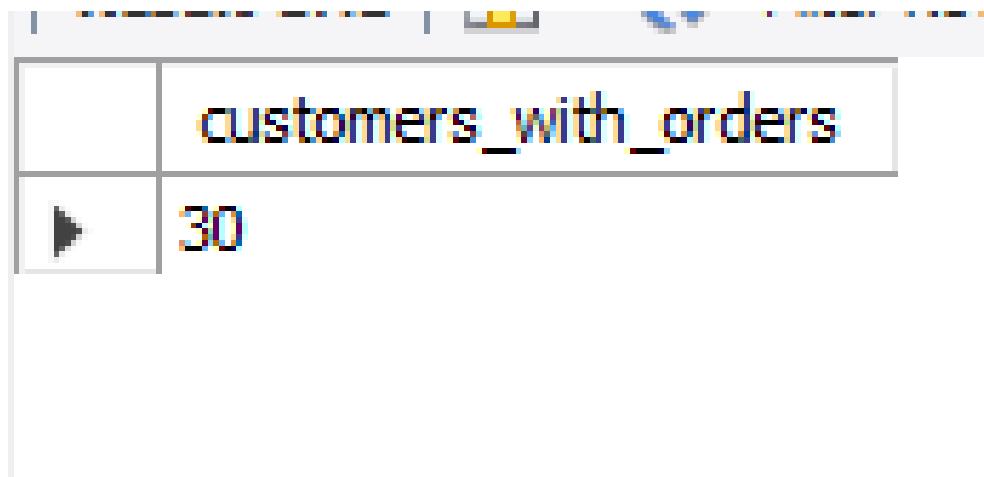
Result Grid		Filter Rows:
	Average Order Value	
▶	17402.43	

4. Count the number of customers who have placed at least one order

SQL Query

```
SELECT  
    COUNT(DISTINCT customer_id) AS  
customers_with_orders  
FROM orders;
```

OUTPUT



The screenshot shows a SQL query execution window with a table of results. The table has two columns: the first column contains a right-pointing triangle icon, and the second column contains the value 30. The table is titled 'customers_with_orders'.

	customers_with_orders
▶	30

5. Find the number of orders placed by each customer

SQL Query

```
SELECT  
    customer_id,  
    COUNT(order_id) AS total_orders  
FROM orders  
GROUP BY customer_id;
```

OUTPUT

	customer_id	total_orders
▶	1	12
	2	17
	3	17
	4	9
	5	14
	6	16
	7	13
	8	10
	9	10
	10	13
	11	9
	12	11
	13	12
	14	15
	15	15
	16	16
	17	17
	18	9

6. Find total sales amount made by each custome

SQL Query

```
SELECT  
    customer_id,  
    SUM(total_amount) AS total_sales  
FROM orders  
GROUP BY customer_id;
```

OUTPUT

Result Grid			Filter Rows:	
	customer_id	total_sales		
▶	1	183747.44		
	2	284420.07		
	3	253783.31		
	4	137562.22		
	5	262504.19		
	6	212173.58		
	7	167960.11		
	8	164701.78		
	9	106226.03		
	10	252722.75		
	11	173409.01		
	12	169653.79		
	13	152727.70		
	14	360324.18		
	15	260499.91		
	16	278469.73		
	17	262189.71		
	18	199823.08		

7. List the number of products sold per category

SQL Query

```
SELECT
    p.category,
    SUM(oi.quantity) AS total_products_sold
FROM products p
JOIN order_items oi
    ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY total_products_sold DESC;
```

OUTPUT

	category	total_products_sold
▶	Electronics	687
	Clothing	559
	Home	443
	Toys	405
	Books	350

8. Find the average item price per category

SQL Query

```
SELECT  
    category,  
    AVG(price) AS average_price  
FROM products  
GROUP BY category  
ORDER BY average_price DESC;
```

OUTPUT

	category	average_price
▶	Clothing	3434.581538
	Books	3167.084286
	Electronics	2653.388571
	Toys	2516.526250
	Home	2146.367500

9. Show number of orders placed per day

SQL Query

```
SELECT
    DATE(order_date) AS order_date,
    COUNT(order_id) AS total_orders
FROM orders
GROUP BY DATE(order_date)
ORDER BY order_date;
```

OUTPUT

	order_date	total_orders
▶	2024-06-15	1
	2024-06-18	1
	2024-06-19	3
	2024-06-20	2
	2024-06-21	2
	2024-06-23	2
	2024-06-24	1
	2024-06-25	1
	2024-06-30	2
	2024-07-03	1
	2024-07-04	1
	2024-07-05	3
	2024-07-06	2
	2024-07-08	2
	2024-07-09	2
	2024-07-12	1
	2024-07-14	1
	2024-07-16	2

10. List total payments received per payment method

SQL Query

```
SELECT  
    method AS payment_method,  
    SUM(amount_paid) AS  
total_payments  
FROM payments  
GROUP BY method  
ORDER BY total_payments DESC;
```

OUTPUT

Result Grid			Filter Rows:
	payment_method	total_payments	
▶	Debit Card	1930577.88	
	Credit Card	1754603.10	
	Net Banking	1658383.90	
	UPI	1617408.78	

Level 4: Multi-Table Queries (JOINS)

1. Retrieve order details along with the customer name (INNER JOIN)

SQL Query

```
SELECT
    o.order_id,
    c.customer_id,
    c.name AS customer_name,
    o.order_date,
    o.status,
    o.total_amount
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id;
```

OUTPUT

	order_id	customer_id	customer_name	order_date	status	total_amount
▶	14	1	Thomas Owens	2024-09-24 21:21:38	Shipped	15803.34
	17	1	Thomas Owens	2024-08-19 21:17:57	Pending	11173.77
	61	1	Thomas Owens	2024-12-26 23:21:58	Shipped	13053.00
	76	1	Thomas Owens	2025-01-14 22:59:54	Pending	23506.81
	92	1	Thomas Owens	2024-09-26 11:33:58	Shipped	834.75
	109	1	Thomas Owens	2025-03-17 04:56:18	Shipped	12190.14
	127	1	Thomas Owens	2025-06-10 18:01:32	Pending	20160.64
	135	1	Thomas Owens	2025-03-11 22:58:23	Pending	8891.79
	144	1	Thomas Owens	2024-09-19 09:18:22	Shipped	12093.54
	221	1	Thomas Owens	2024-09-25 05:49:39	Pending	12437.61
	278	1	Thomas Owens	2025-02-01 20:05:38	Delivered	32015.16
	379	1	Thomas Owens	2025-03-22 15:57:58	Cancelled	21586.89
	56	2	Charles Grant	2024-06-23 15:59:57	Pending	6828.61
	169	2	Charles Grant	2025-02-11 04:04:38	Cancelled	9426.30
	209	2	Charles Grant	2024-10-16 10:07:48	Shipped	7690.62
	223	2	Charles Grant	2024-11-25 01:32:18	Cancelled	41020.75
	230	2	Charles Grant	2024-07-25 22:19:17	Pending	22655.32
	232	2	Charles Grant	2025-05-02 17:14:21	Cancelled	9583.84
	242	2	Charles Grant	2025-06-06 14:06:45	Pending	8888.58

3. List all orders with their payment method (INNER JOIN)

SQL Query

```
SELECT
    o.order_id,
    o.order_date,
    o.status,
    o.total_amount,
    p.payment_id,
    p.method AS payment_method,
    p.amount_paid,
    p.payment_date
FROM orders o
INNER JOIN payments p
    ON o.order_id = p.order_id;
```

OUTPUT

Result Grid		Filter Rows:		Export:	Wrap Cell Content: ⌵			
	order_id	order_date	status	total_amount	payment_id	payment_method	amount_paid	payment_date
▶	1	2025-03-02 07:20:11	Delivered	9414.28	1	Credit Card	9414.28	2025-03-02 08:16:11
	2	2024-10-09 18:08:21	Shipped	532.20	2	Net Banking	532.20	2024-10-09 19:00:21
	3	2025-05-08 00:08:27	Cancelled	5164.56	3	Credit Card	5164.56	2025-05-08 00:55:27
	4	2024-09-19 22:16:13	Delivered	9469.78	4	UPI	9469.78	2024-09-19 22:28:13
	5	2025-04-08 18:02:06	Pending	14501.86	5	UPI	14501.86	2025-04-08 18:26:06
	6	2024-10-25 07:33:59	Cancelled	31050.17	6	UPI	31050.17	2024-10-25 08:16:59
	7	2024-07-29 11:58:47	Shipped	3043.67	7	UPI	3043.67	2024-07-29 12:45:47
	8	2024-07-30 22:49:49	Cancelled	32714.06	8	Net Banking	32714.06	2024-07-30 23:42:49
	9	2025-06-10 17:00:25	Pending	24219.20	9	Net Banking	24219.20	2025-06-10 17:36:25
	10	2025-02-16 12:45:59	Delivered	24342.52	10	Debit Card	24342.52	2025-02-16 13:43:59
	11	2025-03-11 14:22:46	Cancelled	16196.13	11	Credit Card	16196.13	2025-03-11 15:05:46
	12	2025-01-15 13:22:53	Shipped	9890.72	12	Credit Card	9890.72	2025-01-15 14:00:53
	13	2025-05-12 13:04:29	Cancelled	9627.36	13	UPI	9627.36	2025-05-12 14:01:29
	14	2024-09-24 21:21:38	Shipped	15803.34	14	UPI	15803.34	2024-09-24 22:11:38
	15	2024-07-18 03:18:24	Pending	12421.88	15	UPI	12421.88	2024-07-18 03:45:24
	16	2024-09-09 03:00:40	Shipped	22856.73	16	UPI	22856.73	2024-09-09 03:38:40
	17	2024-08-19 21:17:57	Pending	11173.77	17	UPI	11173.77	2024-08-19 21:49:57
	18	2024-06-15 12:57:04	Cancelled	32001.24	18	Net Banking	32001.24	2024-06-15 13:45:04
	19	2024-07-26 15:22:24	Cancelled	242.46	19	Net Banking	242.46	2024-07-26 15:42:24

4. Get list of customers and their orders (LEFT JOIN)

SQL Query

```
SELECT
    c.customer_id,
    c.name AS customer_name,
    c.email,
    o.order_id,
    o.order_date,
    o.status,
    o.total_amount
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
ORDER BY c.customer_id;
```

OUTPUT

Result Grid							
		Filter Rows:		Export:	Wrap Cell Content:		
	customer_id	customer_name	email	order_id	order_date	status	total_amount
▶	1	Thomas Owens	user1@example.com	14	2024-09-24 21:21:38	Shipped	15803.34
	1	Thomas Owens	user1@example.com	17	2024-08-19 21:17:57	Pending	11173.77
	1	Thomas Owens	user1@example.com	61	2024-12-26 23:21:58	Shipped	13053.00
	1	Thomas Owens	user1@example.com	76	2025-01-14 22:59:54	Pending	23506.81
	1	Thomas Owens	user1@example.com	92	2024-09-26 11:33:58	Shipped	834.75
	1	Thomas Owens	user1@example.com	109	2025-03-17 04:56:18	Shipped	12190.14
	1	Thomas Owens	user1@example.com	127	2025-06-10 18:01:32	Pending	20160.64
	1	Thomas Owens	user1@example.com	135	2025-03-11 22:58:23	Pending	8891.79
	1	Thomas Owens	user1@example.com	144	2024-09-19 09:18:22	Shipped	12093.54
	1	Thomas Owens	user1@example.com	221	2024-09-25 05:49:39	Pending	12437.61
	1	Thomas Owens	user1@example.com	278	2025-02-01 20:05:38	Delivered	32015.16
	1	Thomas Owens	user1@example.com	379	2025-03-22 15:57:58	Cancelled	21586.89
	2	Charles Grant	user2@example.com	56	2024-06-23 15:59:57	Pending	6828.61
	2	Charles Grant	user2@example.com	169	2025-02-11 04:04:38	Cancelled	9426.30
	2	Charles Grant	user2@example.com	209	2024-10-16 10:07:48	Shipped	7690.62
	2	Charles Grant	user2@example.com	223	2024-11-25 01:32:18	Cancelled	41020.75
	2	Charles Grant	user2@example.com	230	2024-07-25 22:19:17	Pending	22655.32
	2	Charles Grant	user2@example.com	232	2025-05-02 17:14:21	Cancelled	9583.84

5. List all products along with order item quantity (LEFT JOIN)

SQL Query

```
SELECT
    p.product_id,
    p.name AS product_name,
    p.category,
    p.price,
    oi.quantity
FROM products p
LEFT JOIN order_items oi
    ON p.product_id = oi.product_id
ORDER BY p.product_id;
```

OUTPUT

	product_id	product_name	category	price	quantity
▶	1	Plant No	Home	639.43	1
	1	Plant No	Home	639.43	3
	1	Plant No	Home	639.43	1
	1	Plant No	Home	639.43	1
	1	Plant No	Home	639.43	2
	1	Plant No	Home	639.43	3
	1	Plant No	Home	639.43	2
	1	Plant No	Home	639.43	2
	1	Plant No	Home	639.43	2
	1	Plant No	Home	639.43	2
	1	Plant No	Home	639.43	2
	1	Plant No	Home	639.43	3
	1	Plant No	Home	639.43	3
	1	Plant No	Home	639.43	2
	1	Plant No	Home	639.43	2
	1	Plant No	Home	639.43	2
	1	Plant No	Home	639.43	3
	1	Plant No	Home	639.43	1

6. List all payments including those with no matching orders (RIGHT JOIN)

SQL Query

```
SELECT
  o.order_id,
  o.order_date,
  o.status,
  p.payment_id,
  p.payment_date,
  p.method AS payment_method,
  p.amount_paid
FROM orders o
RIGHT JOIN payments p
  ON o.order_id = p.order_id
ORDER BY p.payment_id;
```

OUTPUT

Result Grid		Filter Rows:		Export:	Wrap Cell Content:		
	order_id	order_date	status	payment_id	payment_date	payment_method	amount_paid
▶	1	2025-03-02 07:20:11	Delivered	1	2025-03-02 08:16:11	Credit Card	9414.28
	2	2024-10-09 18:08:21	Shipped	2	2024-10-09 19:00:21	Net Banking	532.20
	3	2025-05-08 00:08:27	Cancelled	3	2025-05-08 00:55:27	Credit Card	5164.56
	4	2024-09-19 22:16:13	Delivered	4	2024-09-19 22:28:13	UPI	9469.78
	5	2025-04-08 18:02:06	Pending	5	2025-04-08 18:26:06	UPI	14501.86
	6	2024-10-25 07:33:59	Cancelled	6	2024-10-25 08:16:59	UPI	31050.17
	7	2024-07-29 11:58:47	Shipped	7	2024-07-29 12:45:47	UPI	3043.67
	8	2024-07-30 22:49:49	Cancelled	8	2024-07-30 23:42:49	Net Banking	32714.06
	9	2025-06-10 17:00:25	Pending	9	2025-06-10 17:36:25	Net Banking	24219.20
	10	2025-02-16 12:45:59	Delivered	10	2025-02-16 13:43:59	Debit Card	24342.52
	11	2025-03-11 14:22:46	Cancelled	11	2025-03-11 15:05:46	Credit Card	16196.13
	12	2025-01-15 13:22:53	Shipped	12	2025-01-15 14:00:53	Credit Card	9890.72
	13	2025-05-12 13:04:29	Cancelled	13	2025-05-12 14:01:29	UPI	9627.36
	14	2024-09-24 21:21:38	Shipped	14	2024-09-24 22:11:38	UPI	15803.34
	15	2024-07-18 03:18:24	Pending	15	2024-07-18 03:45:24	UPI	12421.68
	16	2024-09-09 03:00:40	Shipped	16	2024-09-09 03:38:40	UPI	22856.73
	17	2024-08-19 21:17:57	Pending	17	2024-08-19 21:49:57	UPI	11173.77
	18	2024-06-15 12:57:04	Cancelled	18	2024-06-15 13:45:04	Net Banking	32001.24
	19	2024-03-06 15:22:24	Cancelled	19	2024-03-06 15:42:24	Net Banking	2421.16

7. Combine data from three tables: customer, order, and payment

SQL Query

```
SELECT
    c.customer_id,
    c.name AS customer_name,
    c.email,
    o.order_id,
    o.order_date,
    o.status,
    o.total_amount,
    p.payment_id,
    p.payment_date,
    p.method AS payment_method,
    p.amount_paid
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
INNER JOIN payments p
    ON o.order_id = p.order_id
ORDER BY o.order_id;
```

OUTPUT

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	order_id	order_date	status	payment_id	payment_date	payment_method	amount_paid
	1	2025-03-02 07:20:11	Delivered	1	2025-03-02 08:16:11	Credit Card	9414.28
	2	2024-10-09 18:08:21	Shipped	2	2024-10-09 19:00:21	Net Banking	532.20
	3	2025-05-08 00:08:27	Cancelled	3	2025-05-08 00:55:27	Credit Card	5164.56
	4	2024-09-19 22:16:13	Delivered	4	2024-09-19 22:28:13	UPI	9469.78
	5	2025-04-08 18:02:06	Pending	5	2025-04-08 18:26:06	UPI	14501.86
	6	2024-10-25 07:33:59	Cancelled	6	2024-10-25 08:16:59	UPI	31050.17
	7	2024-07-29 11:58:47	Shipped	7	2024-07-29 12:45:47	UPI	3043.67
	8	2024-07-30 22:49:49	Cancelled	8	2024-07-30 23:42:49	Net Banking	32714.06
	9	2025-06-10 17:00:25	Pending	9	2025-06-10 17:36:25	Net Banking	24219.20
	10	2025-02-16 12:45:59	Delivered	10	2025-02-16 13:43:59	Debit Card	24342.52
	11	2025-03-11 14:22:46	Cancelled	11	2025-03-11 15:05:46	Credit Card	16196.13
	12	2025-01-15 13:22:53	Shipped	12	2025-01-15 14:00:53	Credit Card	9890.72
	13	2025-05-12 13:04:29	Cancelled	13	2025-05-12 14:01:29	UPI	9627.38
	14	2024-09-24 21:21:38	Shipped	14	2024-09-24 22:11:38	UPI	15803.34
	15	2024-07-18 03:18:24	Pending	15	2024-07-18 03:45:24	UPI	12421.38
	16	2024-09-09 03:00:40	Shipped	16	2024-09-09 03:38:40	UPI	22856.73
	17	2024-08-19 21:17:57	Pending	17	2024-08-19 21:49:57	UPI	11173.77
	18	2024-06-15 12:57:04	Cancelled	18	2024-06-15 13:45:04	Net Banking	32001.24

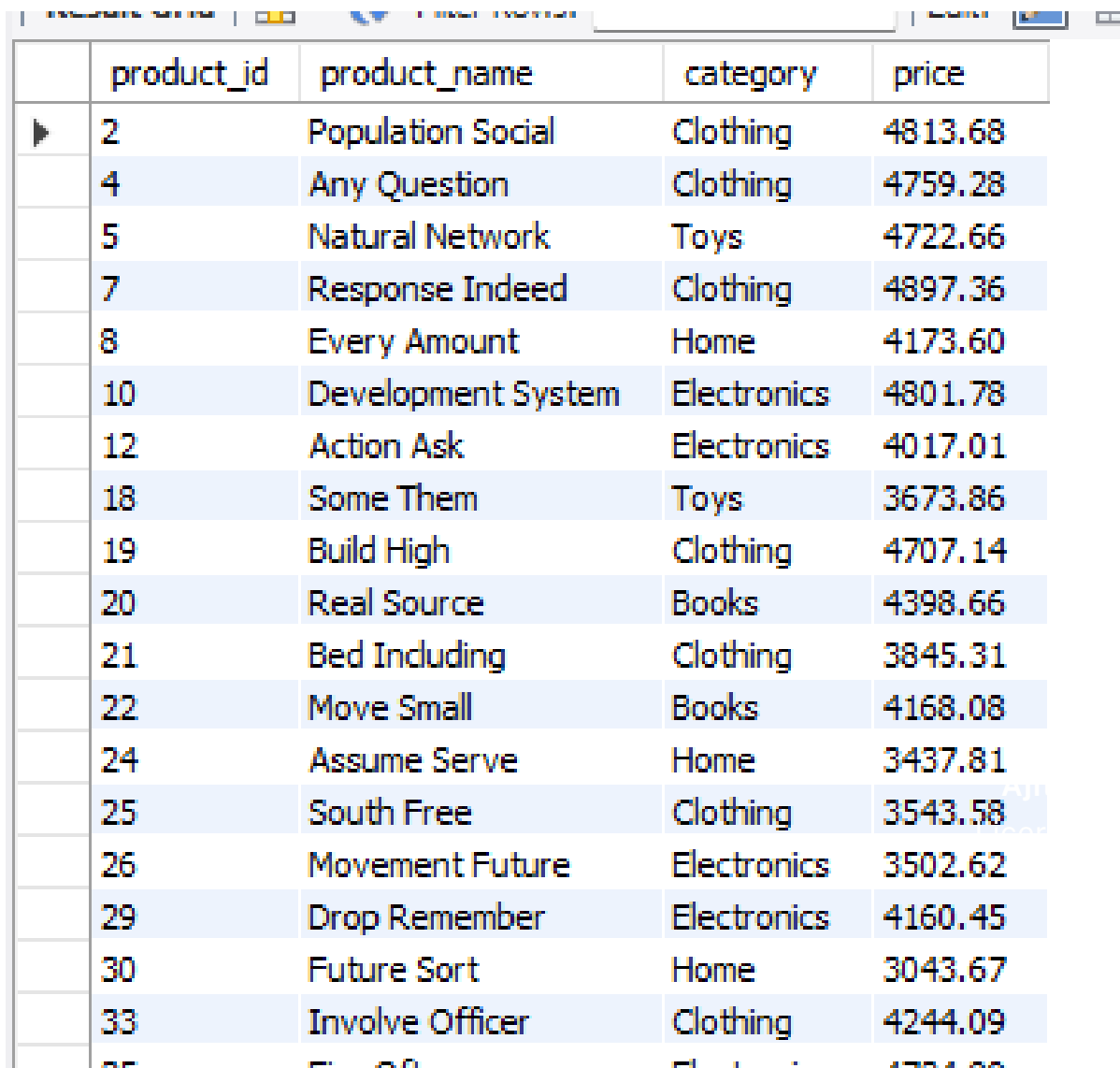
Level 5: Subqueries (Inner Queries)

1. List all products priced above the average product price

SQL Query

```
SELECT
    product_id,
    name AS product_name,
    category,
    price
FROM products
WHERE price > (
    SELECT AVG(price)
    FROM products
);
```

OUTPUT



	product_id	product_name	category	price
▶	2	Population Social	Clothing	4813.68
	4	Any Question	Clothing	4759.28
	5	Natural Network	Toys	4722.66
	7	Response Indeed	Clothing	4897.36
	8	Every Amount	Home	4173.60
	10	Development System	Electronics	4801.78
	12	Action Ask	Electronics	4017.01
	18	Some Them	Toys	3673.86
	19	Build High	Clothing	4707.14
	20	Real Source	Books	4398.66
	21	Bed Including	Clothing	3845.31
	22	Move Small	Books	4168.08
	24	Assume Serve	Home	3437.81
	25	South Free	Clothing	3543.58
	26	Movement Future	Electronics	3502.62
	29	Drop Remember	Electronics	4160.45
	30	Future Sort	Home	3043.67
	33	Involve Officer	Clothing	4244.09
	35	Find So	Electronics	4784.88

2. Find customers who have placed at least one order

SQL Query

```
SELECT
    customer_id,
    name AS customer_name,
    email,
    phone
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
);
```

OUTPUT

Result Grid   Filter Rows: Edit:    Export/Import				
	customer_id	customer_name	email	phone
▶	1	Thomas Owens	user1@example.com	142-479-1945
	2	Charles Grant	user2@example.com	9153947511
	3	Kaitlin Richards	user3@example.com	2073473421
	4	Christina Williams	user4@example.com	586-605-5061x06
	5	David Allen	user5@example.com	(751)456-8289x1
	6	Mark Duke	user6@example.com	(144)957-2811
	7	Briana Wright	user7@example.com	223-833-9635
	8	John Bryan	user8@example.com	045.568.0798x27
	9	Jason Thompson	user9@example.com	1862659420
	10	Shawn Hill	user10@example.com	(268)113-3152x7
	11	Walter Jenkins	user11@example.com	536-329-0817x71
	12	Mary Knight	user12@example.com	361-636-3802
	13	Leslie Wilson	user13@example.com	+1-256-261-1984
	14	Deborah Arias	user14@example.com	811-821-2144x97
	15	Austin Flores	user15@example.com	329.901.1576x66
	16	Amy Landry	user16@example.com	+1-278-019-3748
	17	Randy Mooney	user17@example.com	(158)927-7313x3
	18	Jeffrey Bray	user18@example.com	164.962.0222x92
	19	Thomas Smith	user19@example.com	888.881.8788.88

3. Show orders whose total amount is above the average for that customer

SQL Query

```
SELECT
  o.order_id,
  o.customer_id,
  o.order_date,
  o.total_amount
FROM orders o
WHERE o.total_amount > (
  SELECT AVG(o2.total_amount)
  FROM orders o2
  WHERE o2.customer_id = o.customer_id
);
```

OUTPUT

	order_id	customer_id	order_date	total_amount
▶	6	29	2024-10-25 07:33:59	31050.17
	8	19	2024-07-30 22:49:49	32714.06
	9	6	2025-06-10 17:00:25	24219.20
	10	28	2025-02-16 12:45:59	24342.52
	14	1	2024-09-24 21:21:38	15803.34
	16	5	2024-09-09 03:00:40	22856.73
	18	13	2024-06-15 12:57:04	32001.24
	21	22	2025-03-12 18:53:46	25364.11
	22	20	2025-02-21 11:21:06	21281.44
	24	6	2024-06-19 07:09:04	22882.19
	25	17	2024-07-24 12:56:24	23749.66
	26	24	2025-02-16 23:16:49	35167.92
	29	7	2025-02-17 18:03:18	26308.25
	30	23	2024-10-18 13:37:49	18628.74
	32	15	2024-10-20 06:47:38	23536.35
	33	8	2025-04-27 01:26:11	36147.09
	36	25	2024-07-08 07:43:18	34125.89
	40	26	2024-09-11 08:45:47	24649.49

4. Display customers who haven't placed any orders

SQL Query

```
SELECT
  customer_id,
  name AS customer_name,
  email,
  phone
FROM customers
WHERE customer_id NOT IN (
  SELECT customer_id
  FROM orders
);
```

OUTPUT

Result Grid



Filter Rows:

Edit

	customer_id	customer_name	email	phone
	NULL	NULL	NULL	NULL

5. Show products that were never ordered

SQL Query

```
SELECT
    product_id,
    name AS product_name,
    category,
    price,
    stock_quantity
FROM products
WHERE product_id NOT IN (
    SELECT product_id
    FROM order_items
);
```

OUTPUT

Result Grid |  Filter Rows: | Edit:    | Export/Import: 

	product_id	product_name	category	price	stock_quantity
	NULL	NULL	NULL	NULL	NULL

6. Show highest value order per customer

SQL Query

```
SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.total_amount
FROM orders o
WHERE o.total_amount = (
    SELECT MAX(o2.total_amount)
    FROM orders o2
    WHERE o2.customer_id = o.customer_id
);
```

OUTPUT

Result Grid		Filter Rows:		Edit:
	order_id	customer_id	order_date	total_amount
▶	18	13	2024-06-15 12:57:04	32001.24
	33	8	2025-04-27 01:26:11	36147.09
	62	10	2025-06-13 23:21:01	35723.17
	66	27	2024-06-19 09:41:16	26176.05
	68	24	2025-03-15 07:27:20	36159.92
	80	12	2025-03-28 18:42:12	38656.26
	107	14	2025-02-01 02:02:18	48042.06
	121	16	2024-11-21 16:50:01	37415.67
	128	7	2025-05-05 16:49:18	28589.04
	133	11	2024-11-20 00:44:46	37129.82
	139	18	2025-04-13 20:12:26	39857.19
	174	19	2024-09-30 11:32:38	35676.00
	189	25	2024-12-17 18:53:05	40872.29
	196	21	2025-02-07 10:13:15	24096.64
	198	22	2024-11-11 00:17:56	36504.68
	243	15	2024-12-19 09:20:09	40510.54
	256	17	2025-06-08 16:15:49	40944.10
	258	26	2025-04-18 05:53:20	38836.44
	266	5	2025-02-24 04:25:00	30001.70

7. Highest Order Per Customer (Including Names)

SQL Query

```
SELECT
  c.customer_id,
  c.name AS customer_name,
  o.order_id,
  o.order_date,
  o.total_amount
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.total_amount = (
  SELECT MAX(o2.total_amount)
  FROM orders o2
  WHERE o2.customer_id = o.customer_id
)
ORDER BY c.customer_id;
```

OUTPUT

Result Grid Filter Rows: Export: Wrap Cell Content: A					
	customer_id	customer_name	order_id	order_date	total_amount
▶	1	Thomas Owens	278	2025-02-01 20:05:38	32015.16
	2	Charles Grant	315	2025-01-27 15:23:27	42056.04
	3	Kaitlin Richards	281	2024-11-08 07:40:55	41679.11
	4	Christina Williams	386	2024-12-18 11:35:41	25747.34
	5	David Allen	266	2025-03-21 21:25:00	39921.78
	6	Mark Duke	330	2025-01-10 15:50:45	39003.19
	7	Briana Wright	128	2025-05-05 16:49:18	28589.04
	8	John Bryan	33	2025-04-27 01:26:11	36147.09
	9	Jason Thompson	348	2024-12-29 18:35:10	21414.44
	10	Shawn Hill	62	2025-06-13 23:21:01	35723.17
	11	Walter Jenkins	133	2024-11-20 00:44:46	37129.82
	12	Mary Knight	80	2025-03-28 18:42:12	38656.26
	13	Leslie Wilson	18	2024-06-15 12:57:04	32001.24
	14	Deborah Arias	107	2025-02-01 02:02:18	48042.06
	15	Austin Flores	243	2024-12-19 09:20:09	40510.54
	16	Amy Landry	121	2024-11-21 16:50:01	37415.67
	17	Randy Mooney	256	2025-06-08 16:15:49	40944.10
	18	Jeffrey Bray	139	2025-04-13 20:12:26	39857.19
	19	Anna Smith	174	2024-08-28 11:02:00	35576.00

Level 6: Set Operations

1. List all customers who have either placed an order or written a product review

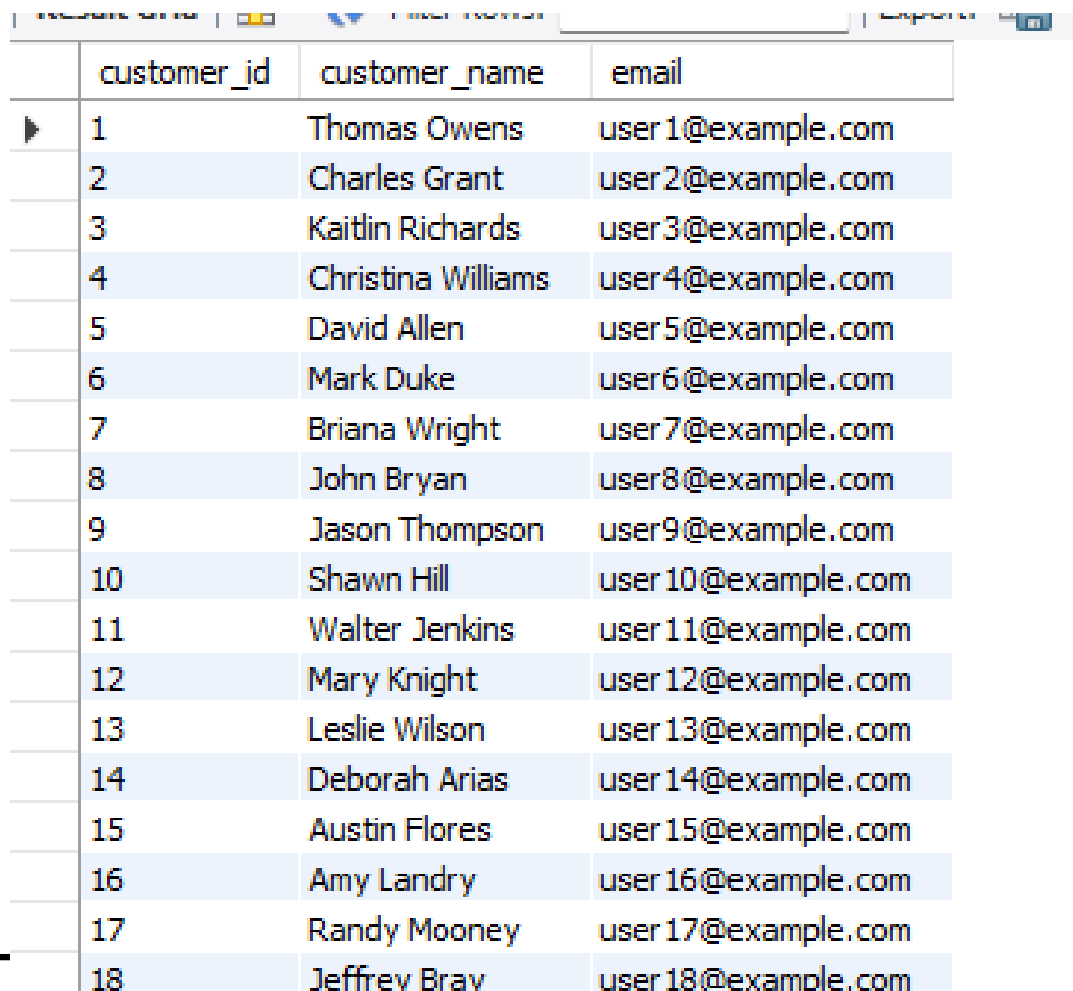
SQL Query

```
SELECT
  c.customer_id,
  c.name AS customer_name,
  c.email
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id

UNION

SELECT
  c.customer_id,
  c.name AS customer_name,
  c.email
FROM customers c
JOIN product_reviews pr
  ON c.customer_id = pr.customer_id;
```

OUTPUT



The screenshot shows a database application window with a table of customer data. The table has four columns: an index, customer_id, customer_name, and email. There are 18 rows of data, each representing a customer. The interface includes a toolbar with icons for various database operations and a status bar at the bottom.

	customer_id	customer_name	email
▶	1	Thomas Owens	user1@example.com
	2	Charles Grant	user2@example.com
	3	Kaitlin Richards	user3@example.com
	4	Christina Williams	user4@example.com
	5	David Allen	user5@example.com
	6	Mark Duke	user6@example.com
	7	Briana Wright	user7@example.com
	8	John Bryan	user8@example.com
	9	Jason Thompson	user9@example.com
	10	Shawn Hill	user10@example.com
	11	Walter Jenkins	user11@example.com
	12	Mary Knight	user12@example.com
	13	Leslie Wilson	user13@example.com
	14	Deborah Arias	user14@example.com
	15	Austin Flores	user15@example.com
	16	Amy Landry	user16@example.com
	17	Randy Mooney	user17@example.com
	18	Jeffrey Bray	user18@example.com

2.List all customers who have placed an order as well as reviewed a product
[intersect not supported]

SQL Query

```
SELECT DISTINCT
  c.customer_id,
  c.name AS customer_name,
  c.email
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
INNER JOIN product_reviews pr
  ON c.customer_id = pr.customer_id;
```

OUTPUT

Result Grid  Filter Rows: Export: 			
	customer_id	customer_name	email
▶	1	Thomas Owens	user1@example.com
	2	Charles Grant	user2@example.com
	4	Christina Williams	user4@example.com
	6	Mark Duke	user6@example.com
	7	Briana Wright	user7@example.com
	9	Jason Thompson	user9@example.com
	10	Shawn Hill	user10@example.com
	11	Walter Jenkins	user11@example.com
	13	Leslie Wilson	user13@example.com
	14	Deborah Arias	user14@example.com
	15	Austin Flores	user15@example.com
	17	Randy Mooney	user17@example.com
	19	Amanda Bright	user19@example.com
	20	Megan Lee	user20@example.com
	21	Jessica Zamora	user21@example.com
	22	Peter Phillips	user22@example.com
	23	Kathryn Mathews	user23@example.com
	25	Cindy Hart	user25@example.com